

Revisão P2
Construção de Compiladores
Prof. Bráulio de Mello

Erickson G. Müller

11 de junho de 2026

Conteúdos

-

1 Tradução dirigida por sintaxe

- $AS \rightarrow$ tradução
 - código
 - * operações básicas
 - * mesmo nível
 - interpretação
 - ações semânticas
 - add dados TS
 - tratamento de erro

1.1 Árvore de derivação anotada

Atributos associados aos símbolos.

Exemplo:

$D ::= var : T$

$T ::= real$

$T' ::= int$

Ações semânticas:

$\{add\ TS(var.nome, T.tipo)\}$

$\{T.tipo = 'r'\}$

$\{T.tipo = 'i'\}$

$var.nome = x_1$

$T.tipo = 'r'$

cadeia: $x_1 : real$

TS

| x1, r

1.2 Esquemas de tradução

Esquemas de tradução é o conjunto de ações semânticas

- extensão da GLC
- atributos

- sintetizados
- herdados
- ações semânticas
- grafo de dependência
 - ordenação uso atributos
- passos
 - Análise sintática
 - Árvore de derivação (anotada)
 - grafo de dependência

Exemplo 2:

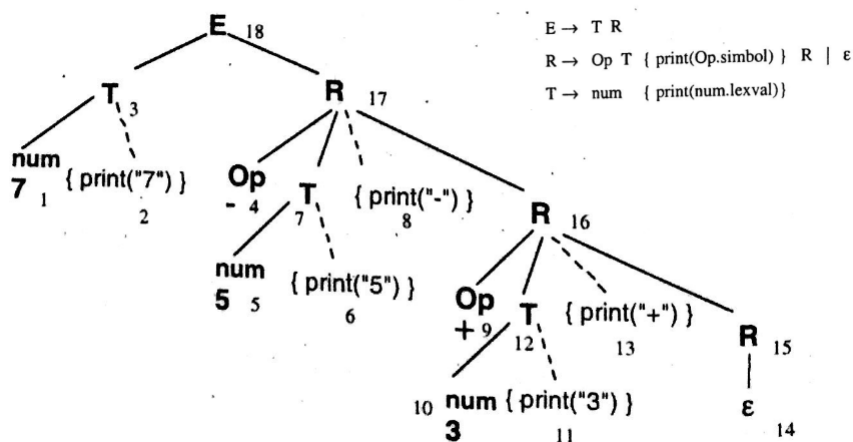
$E ::= TR$

$R ::= Op\ T\ \{\text{print}(Op.\text{simbolo})\}\ R\ |\ \epsilon$

$T ::= num\ \{\text{print}(num.\text{lexval})\}$

4.1.1 Ordem de execução das ações semânticas

Pode-se visualizar a ordem em que as ações semânticas são executadas, desenhando-se a árvore de derivação e agregando-se as ações como se fossem folhas da árvore. Isso vale tanto para ações especificadas à direita das regras de produção, como para ações entremeadas com os símbolos gramaticais. A Figura 4.1 ilustra uma aplicação do esquema do exemplo anterior para a sentença de entrada **7-5+3**. A regra de ordenação é: “as ações são executadas quando as folhas correspondentes são visitadas usando a estratégia *depth-first*” (o algoritmo de caminhamento é apresentado a seguir). Quando a árvore é percorrida na ordem *depth-first*, as ações imprimem a expressão pós-fixada **7 5 - 3 +**.



A aula seguiu explicando o exemplo da página 7 do fragmento do livro no arquivo *3_material...* que está no repositório.

Na GLC:

- Adicionar as produções:

$E ::= E - T$

$T ::= T / F$

e respectivas ações semânticas

- Reconhecer a cadeia:

$8 + ((6/2 - 1) + 2) =$

```

-----
GLC:          Ações semânticas:
L ::= E=      {print (E.val)}
E ::= E1+T    {E.val = E1.val + T.val}
E ::= T       {E.val = T.val}
T ::= T1*F    {T.val = T1.val * F.val}
T ::= F       {T.val = F.val}
F ::= (E)     {F.val = E.val}
F ::= id      {F.val = id.lexval}
E ::= E-T     {E.val = E1.val - T.val}
T ::= T/F     {T.val = T1.val / F.val}

```

Árvore:

E													
id.		id.	id.	id.		id.		id.	id.		id.	id.	
lex		lex	lex	lex		lex		lex	lex		lex	lex	
=8		=(	=(	=6		=2		=1	=)		=2	=)	
8	+	(	(	6	/	2	-	1	)	+	2	)	=

						12								
						L								
						E.val = 12								
/	/					T.val = 4								
						F.val = 4								
			/			E.val = 4								
										\		\		\
						E.val = 2						T.val = 2		
						T.val = 2								
						F.val = 2								
				/		E.val = 2								
									\					
					/	E.val=3			\					
						T.val=3								
E.val=8												F.val = 2		
T.val=8					T.val = 6			\		T.val = 1				
F.val=8					F.val=6		F.val=2		F.val=1					
id.lexval=8					id.lexval=6		id.lexval=2		id.lexval=1			id.lexval=2		
8	+	(	(	6	/	2	-	1	)	+	2	)	=	

1.3 Ações Semânticas

↓→ Tabela de Símbolos.

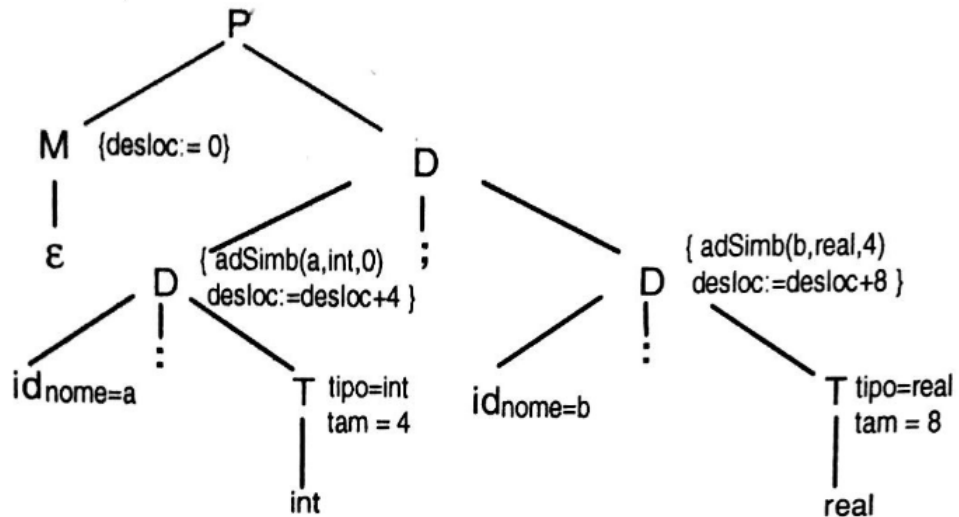
Usamos ações semânticas para adicionar informações na tabela de símbolos. Tradução dirigida pela sintaxe, o conjunto de ações semânticas é chamado de esquemas de tradução.

- esquemas de tradução
 - declarações
 - operações

```

P ::= MD
M ::= \epsilon {desloc = 0}
D ::= D, D
D ::= id:T {addSb (id.nome, T.tipo, desloc); desloc = desloc+T.tam}
T ::= int {T.tipo = int; T.tam=4}
T ::= real {T.tipo = real; t.tam=8}
T ::= array[num] of T1 {T.tipo = matriz(num.val, T1.tipo); T.tam=num.val*T1.tam}
T ::= ^T1 {T.tipo = ponteiro (T1.tipo); T.tam=4}

```



Exemplo de árvore de derivação para "a: int; b: real".

TS

```
-----
| a, int, 0
| b, real, 4
desloc = _ _ 12
         0 4
```

2 Geração de Código Intermediário

A árvore de derivação forma o **segmento de código**.

A grande diferença entre o código intermediário e o código objeto final é que o intermediário não especifica detalhes da máquina alvo, tais como quais registradores serão usados, quais endereços de memória serão referenciados, etc...

- intermediário
- 3 endereços
- independente da máquina
- instruções (arq)

Código intermediário.
Simplifica a implementação do compilador.
Tradução para outras máquinas.
otimização
1 passo a mais.

2.1 Código de 3 endereços

```
A = B op C
A = op B
A = B
goto L
if A oprel B goto L
```

$$A = X + Y * Z$$
$$\begin{matrix} T1 = \\ \cdot \\ \cdot \\ \cdot \\ \cdot \end{matrix}$$

```

S ::= id = E {S.cod=E.cod((gerocod(id.nome='E.nome))}
E ::= E1+E2 {E.nome=geratemp;E.cod=E1.cod((E2.cod(
    (geracod(E.nome='e1.nome'+E2.nome))}
E ::= E1*E2 {E.nome=geratemp;E.cod=E1.cod((E2.cod(
    (geracod(E.nome='e1.nome'+E2.nome))}
E ::= (E1) {E.nome=E1.nome;E.cod=E1.cod}
E ::= id {E.nome=id.nome;E.cod=' '}
```

2.2 Tarefa: gerar esquema de tradução para construir código intermediário adicionando código em saída

$$\begin{aligned} S &::= \text{id} = E \\ E &::= E + T \mid E - T \mid T \\ T &::= T * F \mid T / F \mid F \\ F &::= (E) \mid \text{id} \end{aligned}$$

2.3 Otimização de código

- tempo de compilação

- Desenvolvimento
- Execução de código
- eliminar
 - atribuições de redundância
 - subexpressões comuns
 - temporários desnecessários
- mudar ordem de operações
- gerência de memória

2.4 Representação Blocos Básicos por Grafo Acíclico Dirigido

- folhas: operandos
- nodos: operadores

exemplo: $x = (a + b) * (a - b)$

2.5 Algoritmo de construção de Grafo Acíclico Dirigido

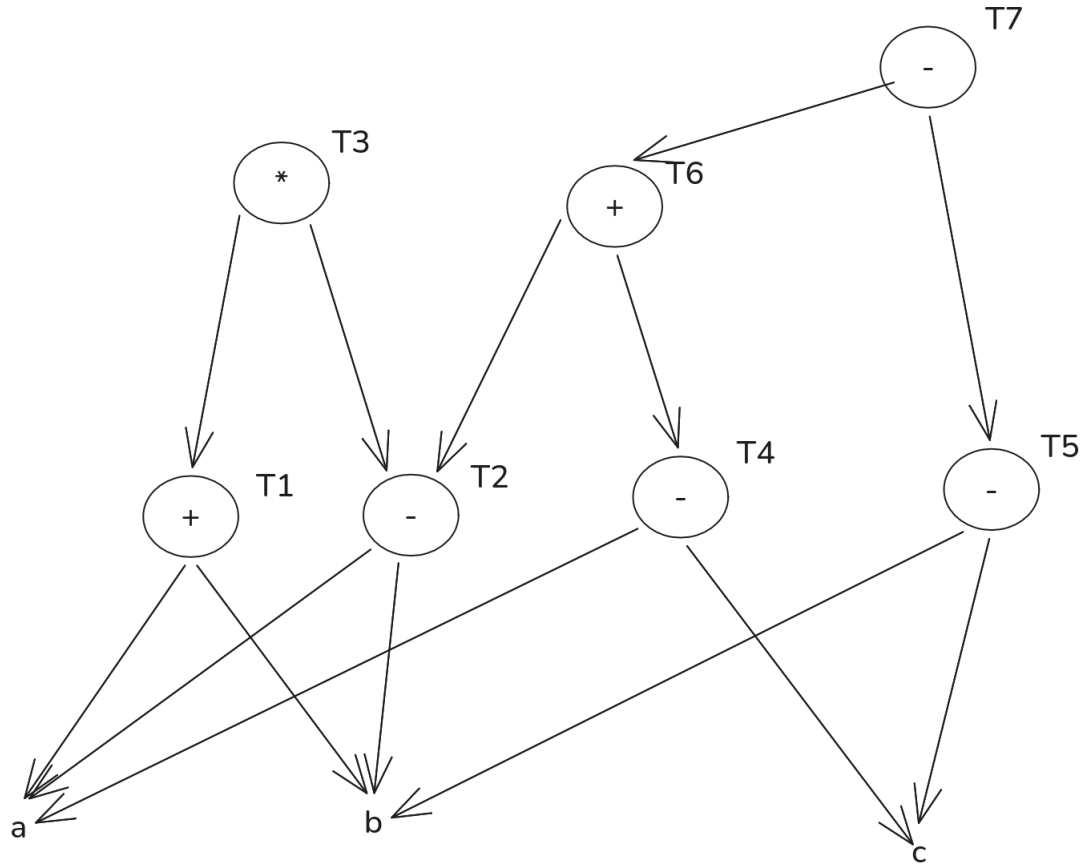
1. Se $y \notin$ no Grafo Acíclico Dirigido then cria folha y
2. se $\exists$ nodo op com folhas y e z then denommine de x else cria nodo x .

Exemplo

```

T1 = a + b
T2 = a - b
T3 = T1 + T2
T4 = a - c
T5 = b - c
T6 = T2 * T4
T7 = T6 - T5

```

2.6 Sequência Otimizada de Execução

- Faça $L = \emptyset$
- Escolha nodo n que $\notin L$
- se $\exists$ arestas incidentes em n then(se originou-se em nodos $\in L$ then *add n in L*)
- se n é o último em L and aresta mais à esquerda originado em n incide em n e $\notin L$ and predecessores de $n \in L$ then add n em L